

PORTADA

Análisis y Diseño de Sistemas II

Semana 7 — Herencia, Polimorfismo e Interfaces

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Un Cuadrado hereda de Rectángulo...
y rompe todo el código que ya funcionaba**

CONTRAEJEMPLO

El clásico problema Cuadrado–Rectángulo

```
class Rectangulo:
    def set_ancho(self, a): self.ancho = a
    def set_alto(self, h): self.alto = h

class Cuadrado(Rectangulo):
    def set_ancho(self, a):
        self.ancho = a
        self.alto = a          # un cuadrado debe medir igual
    def set_alto(self, h):
        self.ancho = h
        self.alto = h

def probar(r: Rectangulo):
    r.set_ancho(5); r.set_alto(4)
    assert r.ancho * r.alto == 20    # falla si r es un Cuadrado
```

Matemáticamente un cuadrado **es un** rectángulo especial. Pero como clase, `Cuadrado` no puede sustituir a `Rectangulo` sin romper el comportamiento que el código que ya existía esperaba.

TRANSICIÓN

Heredar no siempre es la herramienta correcta

El error de `Cuadrado` no está en el álgebra — está en usar **herencia** para modelar una relación que no se comporta como el resto del sistema espera. Hoy cerramos la Unidad 4 viendo tres mecanismos que suelen confundirse: herencia, polimorfismo e interfaces, y cuándo cada uno es la herramienta correcta.

CONCEPTO

Qué es herencia

Herencia es un mecanismo que permite que una clase (subclase) reutilice atributos y comportamiento de otra clase (superclase), y los especialice agregando o redefiniendo lo que necesita. Representa una relación "**es-un**": un `EmpleadoAsalariado` es-un `Empleado` .

Sirve para dos cosas a la vez: **reutilización** (no repetir código común) y **especialización** (agregar comportamiento propio de un caso concreto).

CONCEPTO

Un ejemplo donde sí funciona bien

```
class Empleado:
    def __init__(self, nombre):
        self.nombre = nombre
    def calcular_pago(self):
        raise NotImplementedError

class EmpleadoAsalariado(Empleado):
    def __init__(self, nombre, salario_mensual):
        super().__init__(nombre)
        self.salario_mensual = salario_mensual
    def calcular_pago(self):
        return self.salario_mensual

class EmpleadoPorHoras(Empleado):
    def __init__(self, nombre, horas, tarifa):
        super().__init__(nombre)
        self.horas, self.tarifa = horas, tarifa
    def calcular_pago(self):
        return self.horas * self.tarifa
```

Ambas subclases **son** genuinamente empleados y pueden sustituir a `Empleado` en cualquier parte del sistema sin sorpresas — a diferencia del ejemplo del cuadrado.

CONCEPTO

Cuándo NO usar herencia

- **Cuando la relación no es realmente "es-un"**, sino "tiene-un" o "usa-un" — un `Auto` no es un `Motor` , un auto *tiene* un motor.
- **Cuando solo se quiere reutilizar código**, sin que exista una verdadera relación de especialización entre los conceptos.
- **Cuando heredar rompe el comportamiento esperado** de la superclase, como en `Cuadrado` – `Rectangulo` : la subclase debería poder sustituir a la superclase sin sorpresas, y si no puede, la relación de herencia es la señal equivocada.
- **Cuando el árbol de herencia empieza a crecer solo para acomodar excepciones** (`PatoQueNoVuela` extends `Pato` , con un método vacío que sobrescribe "volar").

CONCEPTO

La alternativa: composición

Composición es construir una clase incluyendo instancias de otras clases como atributos, en vez de heredar de ellas — modela relaciones "tiene-un" en lugar de "es-un".

```
class MotorElectrico:
    def encender(self): ...

class Auto:
    def __init__(self, motor):
        self.motor = motor          # composición, no herencia
    def arrancar(self):
        self.motor.encender()
```

`Auto` no hereda de `MotorElectrico` : lo **usa**. Se puede cambiar el tipo de motor sin tocar la jerarquía de clases de `Auto` .

CONCEPTO

"Preferir composición sobre herencia"

Es una guía de diseño ampliamente aceptada en la industria, no una prohibición absoluta de la herencia: **ante la duda, empezar modelando con composición** suele producir un diseño más flexible que empezar con una jerarquía de clases.

Herencia	Composición
Acopla fuertemente subclase y superclase (cambios en la superclase afectan a todas sus subclases)	Acopla solo a través de la interfaz del objeto compuesto
La relación se fija en tiempo de compilación	Se puede cambiar de componente en tiempo de ejecución
Útil cuando la relación "es-un" es genuina y estable	Útil para reutilizar comportamiento sin comprometerse a una jerarquía rígida

CONCEPTO

Qué es polimorfismo

Polimorfismo (del griego "muchas formas") es la capacidad de tratar objetos de distintas clases a través de una misma interfaz, de modo que el mismo mensaje (llamada a método) produce un comportamiento distinto según el objeto que lo reciba.

El polimorfismo más relevante para el diseño OO es el **polimorfismo de subtipo**: cualquier subclase puede usarse donde se espera su superclase, y cada una responde a su manera.

CONCEPTO

Polimorfismo en acción

```
empleados = [  
    EmpleadoAsalariado("Ana", 3000000),  
    EmpleadoPorHoras("Luis", 80, 25000),  
]  
  
for e in empleados:  
    print(e.nombre, e.calcular_pago())  
    # cada objeto ejecuta SU PROPIA versión  
    # de calcular_pago(), sin que el código  
    # que recorre la lista sepa de qué tipo es cada uno
```

El código que recorre `empleados` no necesita saber si cada elemento es asalariado o por horas — **solo confía en que todos responden a `calcular_pago()`** . Esto es lo que hace posible agregar un tercer tipo de empleado sin tocar este bloque de código.

CONCEPTO

Qué es una interfaz

Una **interfaz** es un contrato: declara qué operaciones debe ofrecer una clase (sus firmas de método), **sin proveer ninguna implementación**. Cualquier clase que "implemente" la interfaz se compromete a cumplir ese contrato, sin heredar código de ella.

```
interface Notificable:
    def notificar(self, mensaje): ...

class NotificadorEmail implements Notificable:
    def notificar(self, mensaje): ...    # su propia implementación

class NotificadorSMS implements Notificable:
    def notificar(self, mensaje): ...    # otra implementación distinta
```

CONCEPTO

Interfaz (contrato) vs. herencia de implementación

Herencia de implementación	Interfaz
La subclase recibe código ya escrito de la superclase	Solo se recibe el compromiso de implementar cada operación
Acopla a la subclase con los detalles internos de la superclase	Acopla únicamente al contrato — cero detalle de implementación compartido
En la mayoría de lenguajes, una clase hereda de una sola superclase	Una clase puede implementar varias interfaces a la vez

Cuando lo único que se necesita es garantizar que varias clases respondan al mismo conjunto de mensajes, una interfaz es un compromiso más liviano que una jerarquía de herencia.

TENSIÓN CONCEPTUAL

¿Por qué no se permite herencia múltiple de clases?

Muchos lenguajes (Java, C#) permiten que una clase implemente **varias interfaces**, pero prohíben heredar de **varias clases** a la vez. La razón: heredar implementación de dos superclases distintas puede generar ambigüedad —si ambas definen el mismo método de forma distinta, ¿cuál gana?—, mientras que combinar varios contratos sin implementación no genera ese conflicto.

Esta asimetría es una de las razones de diseño por las que "preferir interfaces sobre herencia de implementación" es una guía tan repetida en la industria.



INTERACCIÓN

¿Herencia o composición?

En el chat, para cada par, decidan si la relación debería modelarse con herencia ("es-un") o composición ("tiene-un"):

1. GerenteDeVentas y Empleado
2. Pedido y DireccionDeEnvio
3. ReproductorDeMusica y Ecualizador
4. CuentaDeAhorros y CuentaBancaria

5 minutos · respondan en el chat

CONCEPTO · LO QUE VIENE

Sustituir sin sorpresas

El ejemplo de `Cuadrado` – `Rectángulo` con el que abrimos hoy ilustra una idea más general: **toda subclase debería poder sustituir a su clase base en cualquier parte del sistema, sin que el comportamiento se rompa** para el código que ya la estaba usando.

Esa idea se formaliza en la semana 8 como uno de los cinco principios SOLID — hoy la sembramos con intuición de diseñador; ahí le pondremos nombre, autor y reglas precisas.

SÍNTESIS

Ideas clave de hoy

1. **Herencia:** reutilización y especialización a través de una relación "es-un" genuina.
2. No usar herencia solo para reutilizar código, ni cuando la subclase no puede sustituir a la superclase sin sorpresas.
3. **Composición** ("tiene-un") suele ser más flexible que herencia — "preferir composición sobre herencia" es la guía por defecto.
4. **Polimorfismo de subtipo:** mismo mensaje, comportamiento distinto según el objeto que lo recibe.
5. **Interfaces** son un contrato sin implementación — más livianas de acoplar que la herencia de implementación.



CIERRE

Para la próxima semana

Revisen su modelo de dominio y su diagrama de clases: identifiquen si alguna relación de generalización ("es-un") que dibujaron en la Unidad 2 en realidad debería ser una composición, y si hay algún comportamiento compartido que hoy no está expresado como interfaz.

La semana 8 abre la Unidad 5 con los cinco principios SOLID — la formalización de todo lo que hemos construido intuitivamente en abstracción, encapsulamiento, cohesión, acoplamiento, herencia y polimorfismo durante esta semana.

